

Clinical Predictors of PSA: Statistical Modelling and Model Selection in R

Chi Linh Nguyen

Table of contents

0.1	Project aim	2
1	Understand and prepare the data	2
2	Exploratory data analysis	4
2.1	Distribution of log PSA	4
2.2	Cancer volume and log PSA	5
2.3	Prostate weight and log PSA	6
2.4	Log PSA by seminal vesicle invasion status	7
3	Correlation and multicollinearity	8
4	Fit the full multiple linear regression	10
5	Check regression assumptions and influential observations	11
6	Compare the full and reduced models	13
6.1	Reduced model	13
6.2	Nested F-test	14
6.3	AIC and BIC	14
7	Evaluate generalisation with holdout data and cross-validation	15
7.1	Performance on the predefined test set	15
7.2	Ten-fold cross-validation on the training data	15
8	Sensitivity analysis: Numeric versus dummy-coded Gleason score	17
9	Final discussion and limitations	19
9.1	Main findings	19

9.2 Limitations	20
9.3 Conclusion	20

0.1 Project aim

This project investigates how a set of clinical measurements relate to log prostate-specific antigen (lpsa) and how different model-building choices affect predictive performance.

The analysis is deliberately focused on data analysis and statistical modelling, rather than making clinical claims. The dataset is small, observational, and historical, so results are interpreted as statistical associations within this sample rather than causal or current clinical conclusions.

The question is:

Which variables are most informative for log PSA, and how does model complexity affect performance on unseen data?

1 Understand and prepare the data

The dataset contains the following variables:

Variable	Description	Role in main analysis
lcavol	Log cancer volume	Predictor
lweight	Log prostate weight	Predictor
age	Age	Predictor
lbph	Log amount of benign prostatic hyperplasia	Predictor
svi	Seminal vesicle invasion indicator: 0/1	Predictor
lcp	Log capsular penetration	Predictor
gleason	Gleason score	Predictor
pgg45	Percentage of Gleason patterns 4 or 5	Predictor
lpsa	Log PSA	Main continuous outcome
train	Predefined training/test indicator	Data split

The first checks are limited to what is necessary before modelling: dimensions, variable types, missingness, and the predefined split.

```
dim(prostate)
```

```
[1] 97 10
```

```
glimpse(prostate)
```

```
Rows: 97
Columns: 10
$ lcavol <dbl> -0.5798185, -0.9942523, -0.5108256, -1.2039728, 0.7514161, -
1.~
$ lweight <dbl> 2.769459, 3.319626, 2.691243, 3.282789, 3.432373, 3.228826, 3.~
$ age <dbl> 50, 58, 74, 58, 62, 50, 64, 58, 47, 63, 65, 63, 63, 67, 57, 66~
$ lbph <dbl> -1.3862944, -1.3862944, -1.3862944, -1.3862944, -1.3862944, -
1~
$ svi <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,~
$ lcp <dbl> -1.3862944, -1.3862944, -1.3862944, -1.3862944, -1.3862944, -
1~
$ gleason <dbl> 6, 6, 7, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 7, 7, 7, 6, 7, 6, 6, 6,~
$ pgg45 <dbl> 0, 0, 20, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 30, 5, 5, 0, 30, 0, 0, 0,~
$ lpsa <dbl> -0.4307829, -0.1625189, -0.1625189, -0.1625189, 0.3715636, 0.7~
$ train <lgl> TRUE, TRUE, TRUE, TRUE, TRUE, TRUE, FALSE, TRUE, FALSE, FALSE,~
```

```
colSums(is.na(prostate))
```

lcavol	lweight	age	lbph	svi	lcp	gleason	pgg45	lpsa	train
0	0	0	0	0	0	0	0	0	0

```
count(prostate, train)
```

```
# A tibble: 2 x 2
  train     n
  <lgl> <int>
1 FALSE    30
2 TRUE     67
```

The dataset contains 97 observations and 10 variables. There are 67 training observations and 30 test observations.

```
train_data <- prostate |>
  filter(train) |>
  select(-train)

test_data <- prostate |>
  filter(!train) |>
  select(-train)
```

The test set is kept separate from model fitting and used only for evaluation.

```
count(prostate, svi)
```

```
# A tibble: 2 x 2
  svi      n
  <dbl> <int>
1     0    76
2     1    21
```

```
count(prostate, gleason)
```

```
# A tibble: 4 x 2
  gleason      n
  <dbl> <int>
1      6     35
2      7     56
3      8      1
4      9      5
```

`svi` is imbalanced: most observations are 0. `gleason` is also unevenly distributed, with very few observations at the highest levels. This matters later because models using several `gleason` dummy variables may estimate those rare categories imprecisely.

2 Exploratory data analysis

2.1 Distribution of log PSA

```
ggplot(train_data, aes(x = lpsa)) +
  geom_histogram(bins = 15) +
  labs(
    title = "Distribution of log PSA in the training data",
    x = "Log PSA",
    y = "Count"
  )
```



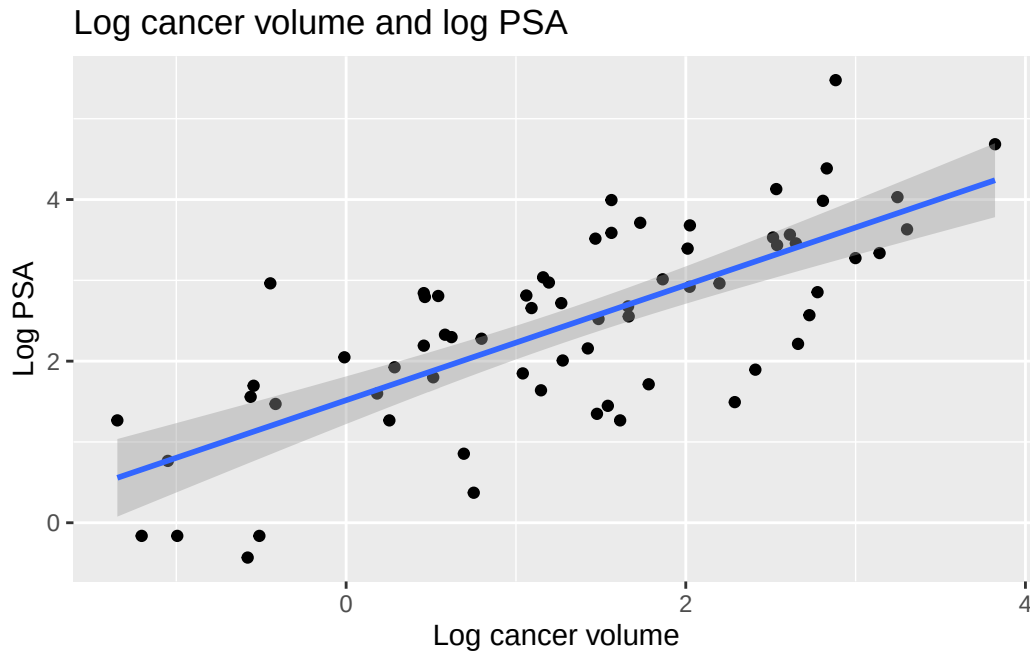
The `lpsa` values are fairly spread out and the distribution is roughly balanced, with no obvious extreme values. The suitability of the linear model is assessed later using the regression diagnostic plots.

2.2 Cancer volume and log PSA

```
ggplot(train_data, aes(x = lcavol, y = lpsa)) +
  geom_point() +
  geom_smooth(method = "lm", se = TRUE) +
  labs(
    title = "Log cancer volume and log PSA",
```

```
x = "Log cancer volume",
y = "Log PSA"
)
```

``geom_smooth()`` using formula = `'y ~ x'`

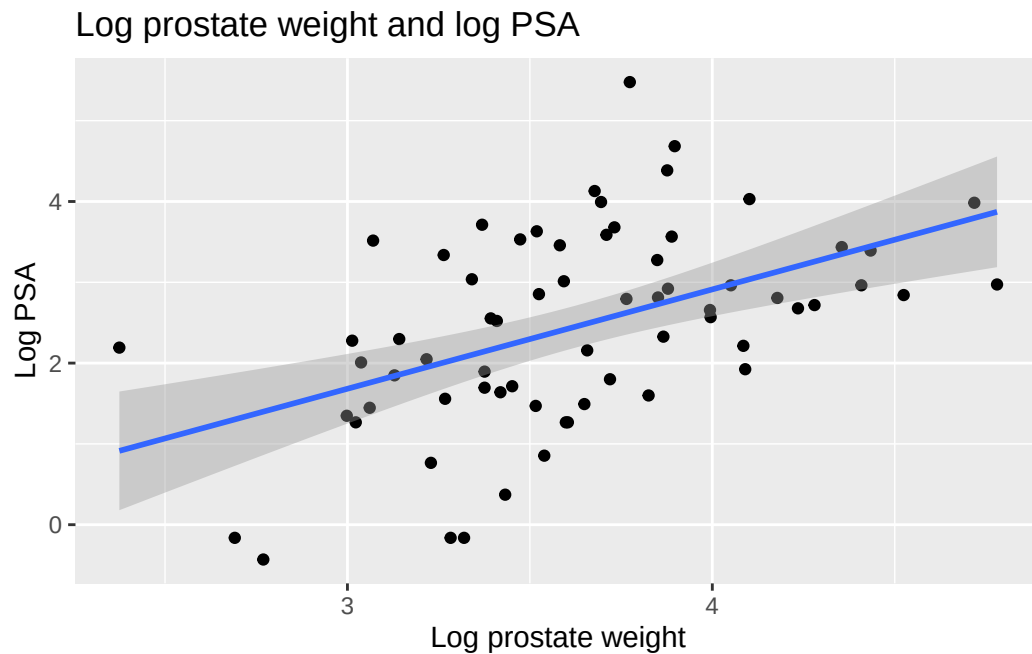


There is a clear positive relationship between `lcavol` and `lpsa`: patients with higher log cancer volume generally have higher log PSA. The relationship is fairly strong in the training data ($r = 0.73$), although there is still some variation around the fitted line.

2.3 Prostate weight and log PSA

```
ggplot(train_data, aes(x = lweight, y = lpsa)) +
  geom_point() +
  geom_smooth(method = "lm", se = TRUE) +
  labs(
    title = "Log prostate weight and log PSA",
    x = "Log prostate weight",
    y = "Log PSA"
  )
```

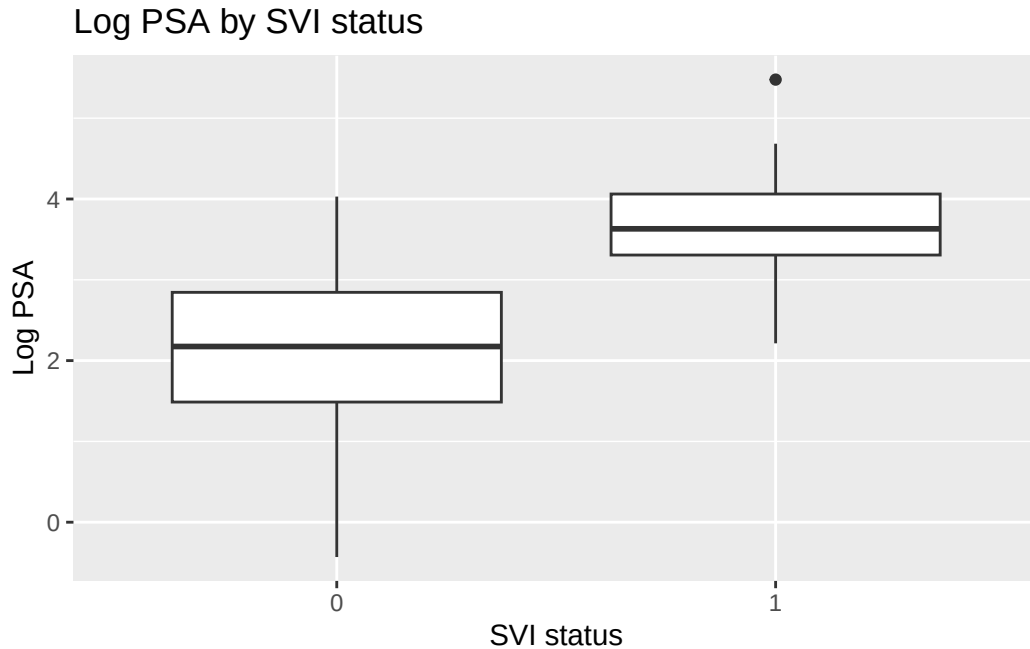
```
`geom_smooth()` using formula = 'y ~ x'
```



`lweight` also has a positive relationship with `lpsa`, but the points are more scattered than for `lcavol`. The correlation is about $r = 0.49$, suggesting a moderate relationship rather than a very strong one.

2.4 Log PSA by seminal vesicle invasion status

```
ggplot(train_data, aes(x = factor(svi), y = lpsa)) +  
  geom_boxplot() +  
  labs(  
    title = "Log PSA by SVI status",  
    x = "SVI status",  
    y = "Log PSA"  
  )
```



The `svi = 1` group has noticeably higher log PSA values than the `svi = 0` group. In the training data, the median `lpsa` is about 3.63 for `svi = 1` compared with 2.17 for `svi = 0`. This is only a descriptive comparison, so the regression model is still needed to see whether `svi` adds information after the other predictors are included.

3 Correlation and multicollinearity

Some predictors are related to similar clinical characteristics, so it is not surprising that they are correlated. The main concern is whether this makes it harder to estimate each predictor's individual effect.

```
predictor_cor <- train_data |>
  select(-lpsa) |>
  cor()

round(predictor_cor, 2)
```

	lcavol	lweight	age	lbph	svi	lcp	gleason	pgg45
lcavol	1.00	0.30	0.29	0.06	0.59	0.69	0.43	0.48
lweight	0.30	1.00	0.32	0.44	0.18	0.16	0.02	0.07

age	0.29	0.32	1.00	0.29	0.13	0.17	0.37	0.28
lbph	0.06	0.44	0.29	1.00	-0.14	-0.09	0.03	-0.03
svi	0.59	0.18	0.13	-0.14	1.00	0.67	0.31	0.48
lcp	0.69	0.16	0.17	-0.09	0.67	1.00	0.48	0.66
gleason	0.43	0.02	0.37	0.03	0.31	0.48	1.00	0.76
pgg45	0.48	0.07	0.28	-0.03	0.48	0.66	0.76	1.00

The strongest relationships in the supplied training data include approximately:

- gleason with pgg45: $r = 0.76$
- lcavol with lcp: $r = 0.69$
- lcp with svi: $r = 0.67$

Because some predictors are strongly correlated, their individual effects may be harder to separate in the regression model.

We can quantify that overlap using variance inflation factors.

```
full_model <- lm(
  lpsa ~ lcavol + lweight + age + lbph +
    svi + lcp + gleason + pgg45,
  data = train_data
)

vif_table <- tibble(
  predictor = names(car::vif(full_model)),
  VIF = as.numeric(car::vif(full_model))
) |>
  arrange(desc(VIF))

vif_table
```

```
# A tibble: 8 x 2
  predictor    VIF
  <chr>      <dbl>
1 pgg45      3.31
2 lcp        3.12
3 gleason    2.64
4 lcavol     2.32
5 svi        2.05
6 lweight    1.47
7 lbph       1.38
8 age        1.36
```

The highest VIF values are about 3.31 for `pgg45` and 3.12 for `lcp`. This shows that some predictors overlap, but none of the VIF values are extremely high. Multicollinearity is therefore worth keeping in mind when interpreting individual coefficients, rather than being a reason to reject the model.

4 Fit the full multiple linear regression

The full model uses all eight predictors to estimate `lpsa`.

```
tidy(full_model, conf.int = TRUE)
```

```
# A tibble: 9 x 7
```

	term <chr>	estimate <dbl>	std.error <dbl>	statistic <dbl>	p.value <dbl>	conf.low <dbl>	conf.high <dbl>
1	(Intercept)	0.429	1.55	0.276	0.783	-2.68	3.54
2	lcavol	0.577	0.107	5.37	0.00000147	0.361	0.792
3	lweight	0.614	0.223	2.75	0.00792	0.167	1.06
4	age	-0.0190	0.0136	-1.40	0.168	-0.0462	0.00825
5	lbph	0.145	0.0705	2.06	0.0443	0.00381	0.286
6	svi	0.737	0.299	2.47	0.0165	0.140	1.33
7	lcp	-0.206	0.111	-1.87	0.0670	-0.428	0.0149
8	gleason	-0.0295	0.201	-0.147	0.884	-0.432	0.373
9	pgg45	0.00947	0.00545	1.74	0.0875	-0.00144	0.0204

```
glance(full_model)
```

```
# A tibble: 1 x 12
```

	r.squared <dbl>	adj.r.squared <dbl>	sigma <dbl>	statistic <dbl>	p.value <dbl>	df <dbl>	logLik <dbl>	AIC <dbl>	BIC <dbl>
1	0.694	0.652	0.712	16.5	2.04e-12	8	-67.5	155.	177.

```
# i 3 more variables: deviance <dbl>, df.residual <int>, nobs <int>
```

The model explains about 69.4% of the variation in `lpsa` in the training data, with an adjusted R^2 of 0.652.

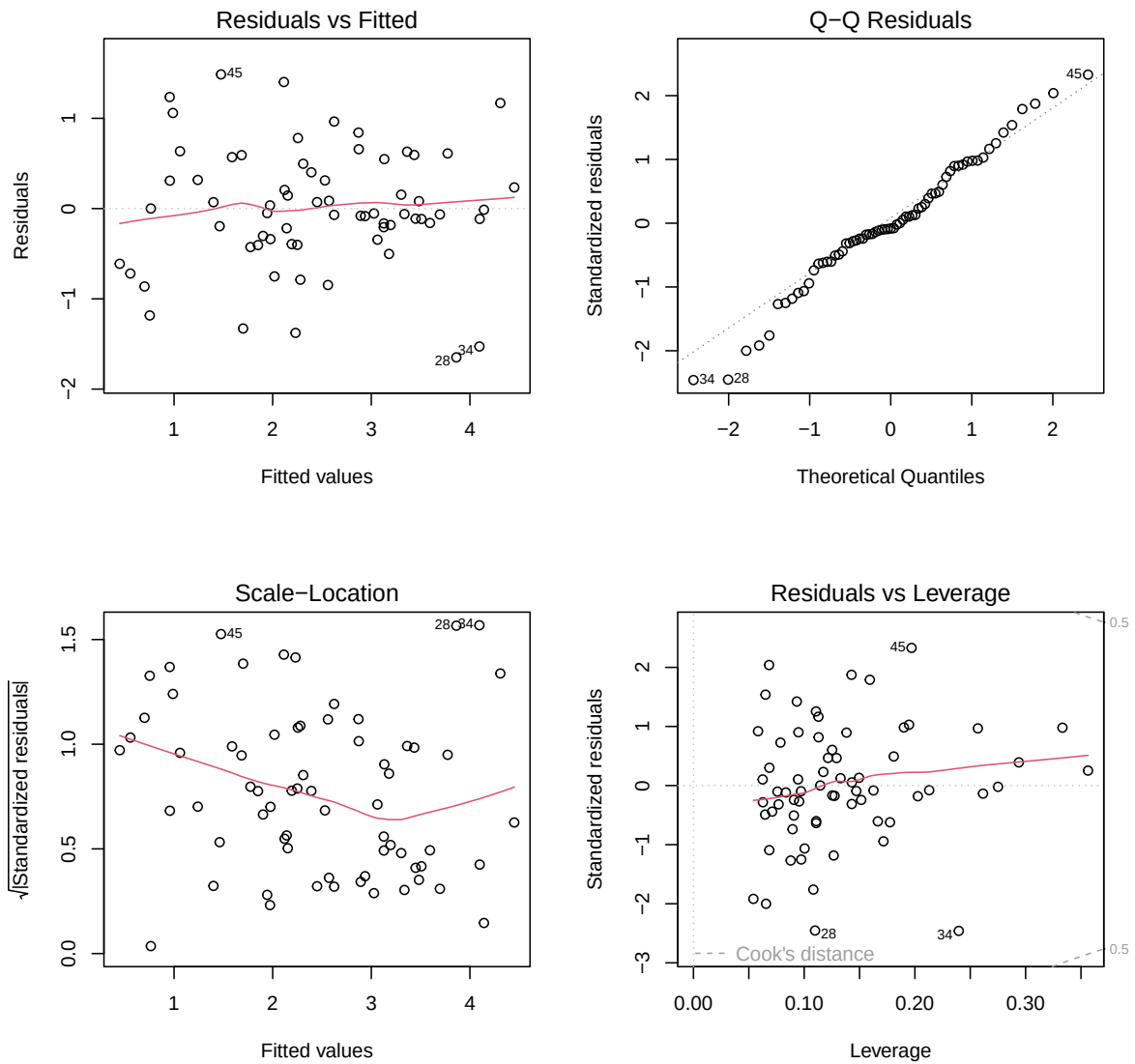
`lcavol` is the clearest predictor in the full model (estimate = 0.58, $p < 0.001$). `lweight`, `lbph`, and `svi` are also significant at the 5% level. In contrast, `age`, `gleason`, `lcp`, and `pgg45` do not show clear additional effects once the other predictors are included.

The result for `gleason` is a useful example of why the correlation analysis matters. It may contain useful information on its own, but it is strongly related to `pgg45` and other predictors,

so the model has more difficulty separating its individual contribution. These coefficients are interpreted as associations within this dataset, not causal effects.

5 Check regression assumptions and influential observations

```
par(mfrow = c(2, 2))  
plot(full_model)
```



```
par(mfrow = c(1, 1))
```

The residual plots do not show a strong curved pattern or a clear funnel shape, and the Q-Q plot is reasonably close to a straight line. Overall, there is no obvious diagnostic problem that would make the linear model unusable, although a few observations stand out more than the rest.

Cook's distance provides a simple screen for influential observations.

```
influence_table <- augment(full_model) |>
  mutate(
    observation = row_number(),
    cook_threshold = 4 / nrow(train_data)
  ) |>
  filter(.cooksad > cook_threshold) |>
  arrange(desc(.cooksad)) |>
  select(observation, .cooksad, .hat, .std.resid)

influence_table
```

```
# A tibble: 5 x 4
  observation .cooksad .hat .std.resid
      <int>    <dbl> <dbl>    <dbl>
1         34  0.212  0.240    -2.46
2         45  0.148  0.197     2.33
3         28  0.0825 0.110    -2.45
4         67  0.0675 0.159     1.79
5         27  0.0650 0.143     1.87
```

Five observations are above the $4/n$ Cook's distance screening threshold. Observations 34 and 45 have the largest Cook's distances, at about 0.21 and 0.15. They may have more influence on the fitted model than most observations, but there is no reason from this check alone to remove them, so they are kept in the analysis.

6 Compare the full and reduced models

6.1 Reduced model

Based on the full model results, a reduced model is fitted using the four predictors that show the clearest evidence of association with `lpsa`: `lcavol`, `lweight`, `lbph`, and `svi`.

```
reduced_model <- lm(
  lpsa ~ lcavol + lweight + lbph + svi,
  data = train_data
)
```

6.2 Nested F-test

```
anova(reduced_model, full_model)
```

Analysis of Variance Table

Model 1: lpsa ~ lcavol + lweight + lbph + svi

Model 2: lpsa ~ lcavol + lweight + age + lbph + svi + lcp + gleason +
pgg45

	Res.Df	RSS	Df	Sum of Sq	F	Pr(>F)
1	62	32.815				
2	58	29.426	4	3.3886	1.6698	0.1693

The nested F-test gives a p-value of about 0.17. This means there is not enough evidence that adding `age`, `lcp`, `gleason`, and `pgg45` as a group improves the four-predictor model at the 5% level. Based on this test, the simpler model with `lcavol`, `lweight`, `lbph`, and `svi` is reasonable.

6.3 AIC and BIC

```
AIC(full_model, reduced_model)
```

	df	AIC
full_model	10	155.0101
reduced_model	6	154.3127

```
BIC(full_model, reduced_model)
```

	df	BIC
full_model	10	177.0570
reduced_model	6	167.5408

The reduced model has a slightly lower AIC than the full model (154.31 vs 155.01), so AIC only gives a small preference to the simpler model. The BIC difference is much larger (167.54 vs 177.06), giving stronger support to the reduced model. This suggests that the extra predictors do not improve the fit enough to clearly justify the added complexity.

7 Evaluate generalisation with holdout data and cross-validation

7.1 Performance on the predefined test set

```
mse <- function(model, newdata) {  
  mean((newdata$lpsa - predict(model, newdata = newdata))^2)  
}  
  
candidate_models <- list(  
  Full = full_model,  
  Reduced = reduced_model  
)  
  
holdout_results <- tibble(  
  model = names(candidate_models),  
  train_MSE = map_dbl(candidate_models, ~ mse(.x, train_data)),  
  test_MSE = map_dbl(candidate_models, ~ mse(.x, test_data))  
) |>  
  arrange(test_MSE)  
  
holdout_results
```

```
# A tibble: 2 x 3  
  model    train_MSE test_MSE  
  <chr>      <dbl>   <dbl>  
1 Reduced    0.490    0.456  
2 Full      0.439    0.521
```

The reduced model has a lower test MSE than the full model (0.456 vs 0.521), even though the full model has a lower training MSE (0.439 vs 0.490). This suggests that the additional predictors improve the fit to the training data but do not improve performance on the holdout test set.

7.2 Ten-fold cross-validation on the training data

A single 30-observation test set can give a noisy ranking. Ten-fold cross-validation gives a second view by repeatedly fitting each fixed candidate formula on different subsets of the 67 training observations.

```

candidate_formulas <- list(
  Full = formula(full_model),
  Reduced = formula(reduced_model)
)

set.seed(123)
folds <- vfold_cv(train_data, v = 10)

fold_mse <- function(split, model_formula) {
  analysis_data <- analysis(split)
  assessment_data <- assessment(split)

  fitted_model <- lm(model_formula, data = analysis_data)
  predictions <- predict(fitted_model, newdata = assessment_data)

  mean((assessment_data$lpsa - predictions)^2)
}

cv_fold_results <- imap_dfr(
  candidate_formulas,
  function(model_formula, model_name) {
    tibble(
      model = model_name,
      fold = seq_along(folds$splits),
      MSE = map_dbl(
        folds$splits,
        ~ fold_mse(.x, model_formula)
      )
    )
  }
)

cv_summary <- cv_fold_results |>
  group_by(model) |>
  summarise(
    mean_CV_MSE = mean(MSE),
    sd_CV_MSE = sd(MSE),
    .groups = "drop"
  ) |>
  arrange(mean_CV_MSE)

cv_summary

```



```
# A tibble: 2 x 3
  model    mean_CV_MSE sd_CV_MSE
  <chr>      <dbl>    <dbl>
1 Full      0.599      0.400
2 Reduced    0.602      0.348
```

The full model has the lowest mean 10-fold cross-validation MSE at approximately 0.599. The full model performs only slightly better than the reduced model in cross-validation (0.599 vs 0.602). The difference is very small, so there is no clear advantage for either model. Together with the holdout result, this suggests that the extra predictors do not provide a consistent improvement in out-of-sample performance.

8 Sensitivity analysis: Numeric versus dummy-coded Gleason score

In the full regression model, `gleason` is treated as a numeric variable. This assumes that moving from Gleason 6 to 7 has the same effect as moving from 7 to 8 or from 8 to 9.

As a sensitivity check, `gleason` is also treated as a factor so that each score level can have its own coefficient. This is an exploratory check and does not change the full-versus-reduced model comparison above.

First, check the number of observations in each Gleason category.

```
count(train_data, gleason)
```

```
# A tibble: 4 x 2
  gleason    n
  <dbl> <int>
1      6    25
2      7    38
3      8     1
4      9     3
```

The training data are very uneven across Gleason categories. Because Gleason 8 and 9 have very few observations, their dummy-variable estimates are likely to be unstable.

To compare the two coding approaches, the same small set of other predictors is used in both models.

```

gleason_levels <- sort(unique(train_data$gleason))

train_gleason <- train_data |>
  mutate(
    gleason_factor = factor(
      gleason,
      levels = gleason_levels
    )
  )

numeric_gleason_model <- lm(
  lpsa ~ lcavol + lweight + lbph + svi + gleason,
  data = train_gleason
)

factor_gleason_model <- lm(
  lpsa ~ lcavol + lweight + lbph + svi + gleason_factor,
  data = train_gleason
)

gleason_comparison <- tibble(
  coding = c("Numeric", "Factor / dummy-coded"),
  AIC = c(
    AIC(numeric_gleason_model),
    AIC(factor_gleason_model)
  )
)

gleason_comparison

```

```

# A tibble: 2 x 2
  coding          AIC
  <chr>          <dbl>
1 Numeric        156.
2 Factor / dummy-coded 157.

```

The numeric model has a lower AIC than the dummy-coded model (approximately 156 vs 157). This suggests that allowing separate coefficients for each Gleason category does not improve the training-data fit enough to justify the additional parameters.

The dummy-coded coefficients can also be inspected directly.

```
tidy(
  factor_gleason_model,
  conf.int = TRUE
) |>
  filter(str_detect(term, "gleason_factor"))
```

```
# A tibble: 3 x 7
  term          estimate std.error statistic p.value conf.low conf.high
<chr>          <dbl>     <dbl>     <dbl>   <dbl>   <dbl>   <dbl>
1 gleason_factor7  0.381      0.227      1.68    0.0980  -0.0725  0.834
2 gleason_factor8  0.0489     0.747     0.0654  0.948   -1.45    1.54
3 gleason_factor9  0.0314     0.466     0.0673  0.947   -0.900   0.963
```

For the dummy-coded model, Gleason 6 is the reference group. Gleason 7 has an estimated coefficient of about 0.38, meaning its estimated `lpsa` is higher than Gleason 6 after controlling for the other predictors. However, the p-value is about 0.098, so the difference is not statistically significant at the 5% level.

The estimates for Gleason 8 and 9 are close to zero and have very large p-values. These estimates are also highly uncertain because there is only one Gleason 8 observation and three Gleason 9 observations in the training data.

Overall, the sensitivity analysis does not provide a strong reason to prefer dummy coding. The numeric model has a lower AIC, while the higher Gleason categories are too sparse to estimate reliably. Therefore, if Gleason is included in the model, treating it as numeric appears to be the simpler choice for this dataset.

9 Final discussion and limitations

9.1 Main findings

1. `lcavol` shows the clearest and most consistent association with log PSA. `lcavol` has a strong positive relationship with `lpsa` in the EDA and remains strongly significant in the full regression (estimate 0.58, $p < 0.001$).
2. Some predictors contain overlapping information, but multicollinearity is not severe. The strongest correlations include `gleason` with `pgg45` ($r = 0.76$) and `lcavol` with `lcp` ($r = 0.69$). The highest VIF is about 3.31, so the overlap should be considered when interpreting coefficients but does not appear extreme.

3. The extra predictors in the full model are not clearly justified. The nested F-test gives $p = 0.169$, so there is not enough evidence that adding `age`, `lcp`, `gleason`, and `pgg45` significantly improves the reduced model. AIC slightly favours the reduced model, while BIC gives stronger support to it.
4. The reduced and full models perform similarly across validation methods. The reduced model performs better on the predefined test set, while the 10-fold cross-validation results are very close between the two models. This suggests that the additional predictors do not give a clear improvement in out-of-sample performance.
5. The Gleason sensitivity analysis does not provide a strong reason to use dummy coding. The numeric model has a lower AIC, while the very small number of Gleason 8 and 9 observations makes the separate category estimates uncertain.

9.2 Limitations

- The dataset is small, with 97 observations in total and only 67 used for model fitting.
- Several predictors are correlated.
- Some Gleason categories are extremely sparse.
- The data are observational, so coefficients should be interpreted as associations rather than causal effects.
- According to the supplied exercise context, the data come from a historical single-hospital cohort of patients who underwent radical prostatectomy. Results therefore should not be assumed to generalise directly to different hospitals, patient groups, or modern clinical practice without external validation.

9.3 Conclusion

Overall, `lcavol` shows the clearest and most consistent association with log PSA in this dataset. The full model fits the training data well, but the nested F-test, AIC and BIC suggest that the extra predictors do not add enough value to justify the added complexity. The validation results also show that model performance is sensitive to how the data are split. Given the small sample size, the reduced model seems to be a reasonable and simpler choice.